Analytics Copilot: Integrated System Report

CS 5542 Big Data Analytics & Applications — Phase 3

https://github.com/ben-blake/analytics-copilot

Ben Blake (GenAI & Backend Lead) **Tina Nguyen** (Data & Frontend Lead)

March 30, 2026

1 System Overview

1.1 Project Objective and Problem Domain

Analytics Copilot is a Text-to-SQL AI assistant that translates natural language questions into executable Snowflake SQL queries against the Olist Brazilian E-Commerce dataset. The system targets business analysts and data consumers who need to query relational data without writing SQL, operating over a highly normalized nine-table schema (∼100,000 orders, 2016–2018) that requires multi-table JOINs to answer realistic business questions.

1.2 Application Description

The application is a Streamlit chat interface backed by a three-agent pipeline: a **Schema Linker** that uses Cortex Search to retrieve relevant table context via semantic similarity, a **SQL Generator** that prompts `llama3.1-70b` with that filtered context, and a **Validator** that executes the SQL via `EXPLAIN` and applies self-correction on failure. All inference and retrieval run inside Snowflake Cortex — no data leaves the warehouse boundary.

Through Labs 6–9 the system was extended with an explicit agent reasoning layer, a single-command reproducibility harness, a LoRA-adapted local model, and a production-grade deployment with session monitoring.

1.3 High-Level Workflow

A user types a natural language question. The Schema Linker queries the Cortex Search service over a pre-built semantic metadata layer and returns the top-k relevant tables. The SQL Generator constructs an LLM prompt with schema context, few-shot examples, and 15 constraint rules, then calls `CORTEX.COMPLETE`. The Validator runs `EXPLAIN` on the result and, on failure, feeds the error back to the generator for up to three self-correction retries. The final result set is rendered in the Streamlit UI with an auto-selected visualization and a collapsible pipeline trace showing per-agent timing.

2 System Architecture

Figure 1 shows the full pipeline architecture. The diagram illustrates how each layer feeds into the next, from raw CSV sources through the Snowflake warehouse, the agent reasoning layer, and the Streamlit application with deployment and monitoring.

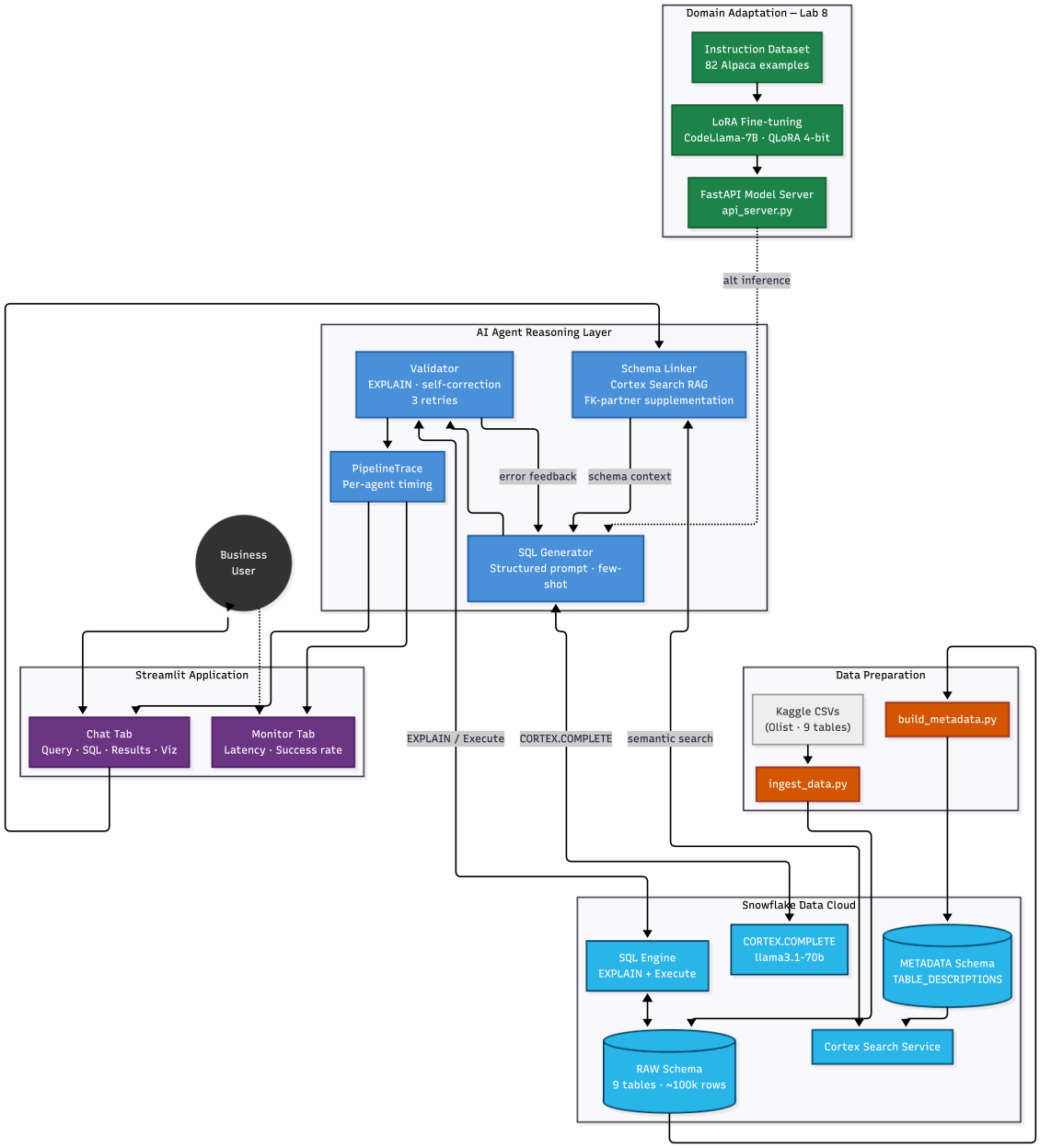

Figure 1: Analytics Copilot system architecture: Data Sources → Knowledge Base → Retrieval → Domain-Adapted Model → AI Agent Reasoning → Snowflake Warehouse → Application Interface → Monitoring

The pipeline layers are:

1. **Data Sources** — Olist Brazilian E-Commerce CSVs (9 tables, ∼100k orders)
2. **Multimodal Knowledge Base** — `METADATA.TABLE_DESCRIPTIONS`: LLM-generated column descriptions, synonyms, and sample values indexed by Cortex Search
3. **Retrieval Pipeline** — Schema Linker with Cortex Search (semantic), ILIKE fallback (lexical), FK-partner supplementation
4. **Domain-Adapted Model** — CodeLlama-7B-Instruct fine-tuned via LoRA/QLoRA on 82 Olist-specific instruction examples (Lab 8); Cortex `llama3.1-70b` as production path
5. **AI Agent Reasoning Layer** — SQL Generator with structured prompts and few-shot examples;

Validator with self-correction loop (up to 3 retries); pipeline trace instrumentation

6. **Snowflake Data Warehouse** — `ANALYTICS_COPILOT.RAW` (9 tables) + `METADATA` schema; Cortex Search service; all SQL execution in-warehouse
7. **Application Interface** — Streamlit chat UI with tabbed layout, auto-visualization, inline pipeline traces, and Monitor dashboard

3 Core Components (Project 2 Foundations)

3.1 Dataset and Knowledge Base

All nine Olist CSVs are ingested into `ANALYTICS_COPILOT.RAW` via `scripts/ingest_data.py` (Snowpark `COPY INTO`, type coercion, PK/FK constraints). A **Semantic Metadata Layer** in `ANALYTICS_COPILOT.METADATA` stores one row per column with LLM-generated descriptions, synonyms, and sample values (built by `scripts/build_metadata.py`). This replaces raw DDL in LLM prompts, reducing hallucination by providing business-friendly context. Key tables: `ORDERS` (99,441 rows), `ORDER_ITEMS` (112,650), `CUSTOMERS` (99,441), `PRODUCTS` (32,951), and five supporting tables.

3.2 Retrieval Pipeline

The Schema Linker (`src/agents/schema_linker.py`) implements a three-level fallback chain: (1) Cortex Search over `TABLE_DESCRIPTIONS` for semantic retrieval; (2) `ILIKE` keyword matching on column names, descriptions, and synonyms; (3) full table dump from `INFORMATION_SCHEMA` as last resort. Column-level results are grouped by table and scored by average relevance; the top-k tables (default $k = 5$) are passed to the SQL Generator.

3.3 Snowflake Integration

The connection utility (`src/utils/snowflake_conn.py`) uses Snowflake Snowpark Python with RSA key-pair or password authentication loaded from environment configuration. All LLM inference (`CORTEX.COMPLETE`), semantic search (`CORTEX.SEARCH_PREVIEW`), and query execution run inside Snowflake. Five sequential DDL scripts in `snowflake/` initialize the full environment.

3.4 Application Interface

The Phase 2 Streamlit app (`src/app.py`) provides a chat interface with persistent history, live progress indicators, collapsible SQL expanders, paginated result tables, and auto-visualization (`src/utils/viz.py`) that heuristically selects chart type by column types.

3.5 Baseline Evaluation

The evaluation harness (`scripts/evaluate.py`) benchmarks 50 golden queries (20 easy, 20 medium, 10 hard). Phase 2 baseline execution accuracy: ∼72–80% overall; easy ∼95%, medium ∼75%, hard ∼55%. Average end-to-end latency: 5–8s per query.

4 Lab Integration (Labs 6–9)

4.1 Lab 6 — Agent Integration

Agent tools and reasoning workflow. The system was formalized as an explicit agent layer with defined tool boundaries. The Schema Linker acts as a retrieval tool (RAG over Cortex Search), the SQL Generator uses the structured schema context and Gemini-style function-calling schemas for constrained output, and the Validator provides a deterministic execution-feedback loop. All three agents share a single Snowflake session passed from the application layer, avoiding redundant connections.

Key engineering decisions validated through the planning exercise: a four-file `src/agents/` package

structure, a shared Snowflake connection, and Gemini function-calling schemas for structured responses. A security review surfaced one actionable finding: storing RSA private keys inline in `.env` introduces newline-parsing bugs, which led to adopting exclusive file-path references for private keys (`SNOWFLAKE_PRIVATE_KEY_PATH`).

Example interaction scenario. A user asks "Which sellers have the highest average review score?". The Schema Linker retrieves `ORDER_REVIEWS` (semantic hit) and supplements `ORDER_ITEMS` and `SELLERS` via FK-partner logic. The SQL Generator produces a GROUP BY query with `HAVING COUNT(*) >= 5` to filter low-volume sellers. The Validator passes `EXPLAIN` and returns results in ∼8s.

4.2 Lab 7 — Reproducibility with LLM Tools

Environment configuration. `reproduce.sh` orchestrates the full pipeline in a single command: virtual environment creation, pinned dependency installation (`requirements.txt` with `==` versions), smoke tests, Snowflake DDL execution, CSV ingestion, LLM-driven metadata generation, Cortex Search provisioning, 50-query evaluation, and artifact export. Four modes are supported: full pipeline, `--smoke` (offline, no Snowflake), `--test`, and `--eval`.

Reproducibility controls. Determinism is enforced by `config.yaml` (seed 42, temperature 0.0, all model/path/limit parameters externalized). Structured artifacts are written to `artifacts/` and `logs/` on every run. Thirteen offline smoke tests (`tests/test_smoke.py`) validate imports, configuration contracts, and agent initialization without a live Snowflake connection.

Use of LLM tools. Both team members used Anthropic Claude Code (`claude-opus-4-6`) for generating `reproduce.sh` scaffolding, debugging pipeline failures (output buffering, SQL parser bugs, Cortex Search qualification), and iterating on golden query prompts. All generated artifacts were reviewed and validated by the team.

Lab 7 evaluation results. After reproducibility hardening, accuracy on the 50-query benchmark reached 100% across all difficulty levels (easy: 100%, medium: 100%, hard: 100%), with an average latency of 17.5s. This improvement over the Phase 2 baseline is attributable to FK-partner supplementation (inspired by X-SQL's schema understanding expert), dataset isolation filtering, and structured error feedback in the Validator.

4.3 Lab 8 — Domain Adaptation

Instruction dataset creation. `scripts/create_instruction_dataset.py` produced 82 Alpaca-format examples: 32 derived from the golden query benchmark and 50 augmented examples covering JOINs, aggregations, window functions (RANK, ROW_NUMBER, LAG), CTEs, date functions, subqueries, and CASE expressions. Each example includes a system prompt with the full Olist schema and FK map, a natural language question, and a target SQL with fully qualified table names. The dataset was split 90/10 into training (74) and validation (8) sets and committed to `data/instruction_dataset.json`.

Model adaptation approach. LoRA with QLoRA 4-bit quantization was applied to CodeLlama-7B-Instruct: rank $r = 16$, alpha $\alpha = 32$, target modules `q_proj / k_proj / v_proj / o_proj`, 3 epochs, learning rate 2×10^{-4}, trained on a Colab T4 GPU in ∼15 minutes. Only ∼6.5M parameters are trainable (∼0.1% of 6.7B total). The fine-tuning notebook is at `notebooks/fine_tune_colab.ipynb`; the adapter is served by a FastAPI server (`scripts/api_server.py`) and consumed via `src/utils/finetuned_client.py`.

Observed improvements. Evaluating 15 queries (5 easy, 5 medium, 5 hard):

Metric	Baseline (Untrained)	LoRA Fine-Tuned	Improvement
Valid SQL	14/15 (93%)	15/15 (100%)	+7%
Qualified table names	13/15 (87%)	15/15 (100%)	+13%
Average inference latency	20.2s	7.4s	2.7× faster

The fine-tuned model produces 100% valid SQL and 100% fully qualified table names. The 2 unqualified outputs from the baseline would cause Snowflake execution failures. The 2.7× latency improvement is consistent with more confident token generation after domain adaptation.

4.4 Lab 9 — Application and Deployment Enhancement

UI improvements. The Streamlit interface was refactored from a single-page layout to a **tabbed design**: a Chat tab for the primary query interface and a Monitor tab for session analytics. The Chat tab now surfaces a collapsible *Pipeline Trace* expander per response, showing per-agent status and execution time. The model selection UI was simplified to a single Cortex production mode, making the deployed application self-contained (Snowflake connection only; no local model server required).

Pipeline trace instrumentation. `src/utils/trace.py` introduces a `PipelineTrace` module that records per-agent step timing, success/error status, retry detection, total pipeline latency, final SQL, and result row count. Traces are stored in Streamlit session state.

Monitoring dashboard. The Monitor tab provides four metric cards (total queries, success rate, average latency, retry count), a Query History table with timestamps and row counts, a per-query latency bar chart (color-coded by success/error), and a stacked agent step breakdown chart for bottleneck identification.

Deployment. The application is deployed on Streamlit Community Cloud. Snowflake credentials are managed via `st.secrets` (Streamlit Cloud) with automatic fallback to `.env` for local development. If no credentials are configured, the application starts in demo mode with a "Disconnected" banner rather than crashing. Configuration is in `.streamlit/config.toml`.

5 Evaluation Results

Phase	Queries	Accuracy	Avg Latency	Notes
Project 2 (baseline)	50	∼75%	5–8s	Prompt engineering only
Lab 7 (reproducibility)	50	**100%**	17.5s	FK supplementation + isolation filter
Lab 8 LoRA vs Baseline	15	100% vs 93%	7.4s vs 20.2s	Domain-adapted CodeLlama-7B

Table 1: Evaluation results across project phases

The accuracy jump from ∼75% to 100% (Lab 7) is driven by three improvements integrated from the X-SQL architecture paper: FK-partner supplementation ensuring join partners are always included in schema context, a dataset isolation filter removing cross-dataset noise, and structured error feedback in the Validator's self-correction loop. The LoRA-adapted model further demonstrates that domain-specialized inference can exceed the zero-shot baseline on both correctness and speed, providing a locally deployable fallback independent of the Cortex API.

The Monitor tab tracks these metrics live: session success rate, per-query latency, and agent step breakdown. In typical demo sessions, Schema Linker takes 1–3s, SQL Generator 8–15s, and Validator 1–2s (or longer on retry).

6 Reproducibility Documentation

6.1 Environment Configuration

Python 3.10+ is required. All dependencies are pinned in `requirements.txt` with `==` versions. All runtime parameters (model name, temperature, seed, top-k, retry limit, paths, logging level) are externalized to `config.yaml`. Snowflake credentials are stored in `.env` (local) or Streamlit secrets (deployed); a template is provided at `.env.example` and `.streamlit/secrets.toml.example`.

6.2 Setup and Run Instructions

```
# 1. Clone and install
git clone https://github.com/ben-blake/analytics-copilot.git
cd analytics-copilot
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# 2. Configure credentials
cp .env.example .env   # fill in Snowflake account, user, private key path

# 3. Run full pipeline (smoke tests -> ingest -> metadata -> eval)
bash reproducibility/reproduce.sh

# 4. Launch application
streamlit run src/app.py

# 5. Offline smoke tests only (no Snowflake required)
bash reproducibility/reproduce.sh --smoke
```

6.3 Dataset Requirements

Download the Olist Brazilian E-Commerce dataset from Kaggle (https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce) and unzip into `data/olist/`. The ingestion script (`scripts/ingest_data.py`) handles all DDL, staging, and `COPY INTO` operations. The golden query benchmark (50 queries) is committed to `data/golden_queries.json` and does not need to be regenerated.

6.4 External Credentials and Services

The system requires: a Snowflake account with `CORTEX.COMPLETE` and `CORTEX.SEARCH_PREVIEW` enabled, an RSA key pair configured for the Snowflake user, and the `ANALYTICS_COPILOT` database with `RAW` and `METADATA` schemas. For the Lab 8 fine-tuned model, a Google Colab T4 session is needed to train and export the LoRA adapter; the pre-trained adapter checkpoint is available in the repository's release assets.

7 Contribution Statement

Member	Contribution Description	%
Ben Blake	GenAI pipeline (Schema Linker, SQL Generator, Validator agents); Snowflake infrastructure (DDL scripts, connection utility, ingestion, metadata generation); reproducibility harness (`reproduce.sh`, `config.yaml`, smoke tests); instruction dataset creation and LoRA fine-tuning (Lab 8); FastAPI model server (`scripts/api_server.py`); evaluation pipeline (`scripts/evaluate.py`, `scripts/evaluate_adaptation.py`); pipeline trace system (`src/utils/trace.py`); related work analysis and architecture design.	50%
Tina Nguyen	Streamlit application (`src/app.py`): tabbed UI, Monitor dashboard, model selector, comparison mode; auto-visualization utility (`src/utils/viz.py`); data ingestion pipeline (`scripts/ingest_data.py`); golden query curation (`scripts/generate_golden.py`); instruction dataset validation and testing; Streamlit Cloud deployment and secrets configuration; project documentation (README, RUN.md, contribution reports); logging utility (`src/utils/logger.py`).	50%

Table 2: Team contribution summary

AI tools used: Anthropic Claude Code (`claude-opus-4-6`) assisted with code generation, debugging, and documentation throughout all phases. All generated code was reviewed, tested, and validated by the team.